

Lecture 08

Lexical Scope, Closures

Group Work

1. For each function binding, write down the enclosing environment.
2. What do each of the following evaluate to?
 - `f 10`
 - `g 10`
 - `h 10`

```
let x = 1
```

```
let y = 2
```

```
let z = 3
```

```
let f = fun x -> x + x
```

```
let g = fun x -> x + y
```

```
let h = fun a -> f x + g y + z * a
```

Lexical Scope

- Key concept: must “get this” for homework and competency
- We already know function bodies can use anything in scope...
 - But now function are getting passed around / returned! “In scope” where??
- **Function bodies evaluate in the environment where defined!**
 - **NOT in the environment where the function is called!**
- Today: examples and why lexical scope is “the right thing”
 - Later in quarter: how to implement lexical scope

Lexical Scope

- Key concept: must “get this” for homework and competency
- We already know function bodies can use anything in scope
 - But now function are getting passed around where??
- **Function bodies evaluate in the environment where defined!**
 - **NOT in the environment where the function is called!**
- Today: examples and why lexical scope is “the right thing”
 - Later in quarter: how to implement lexical

Lexical Scope 😇

Dynamic Scope 😈

Example

```
(* 1 *) let x = 1
(* 2 *) let f y = x + y
(* 3 *) let x = 2
(* 4 *) let y = 3
(* 5 *) let z = f (x + y)
```

- Line 2 defines function **f** which
 - When called, evaluates **x + y**
in an environment where **x** $\mapsto$ **1** and **y** $\mapsto$ **arg**
- Call on Line 5:
 - Looks up **f** to get definition from line 2
 - Evaluates argument expression **(x + y)** in current environment
 - Calls function with argument **5**, result is **6**

Closures

- How can functions be evaluated in old environments?
 - OCaml stores old environments as needed!
- Refined, “official semantics” of functions:
 - A function value has two parts:
 - The **code**
 - The **environment** from when the function was defined
 - This is a “pair”, but hidden from the programmer (no `fst` / `snd`)
 - All we can do is “call with an argument”
 - The “pair” is called a **closure**
 - Calls are evaluated in the closure’s environment extended by parameter $\mapsto$ argument mapping

Example Redux

```
(* 1 *) let x = 1
(* 2 *) let f y = x + y
(* 3 *) let x = 2
(* 4 *) let y = 3
(* 5 *) let z = f (x + y)
```

- Line 2 creates a **closure** and binds **f** to it
 - **Code**: argument **y** and body **x + y**
 - **Environment**: **x** $\mapsto$ **1** (plus anything else in scope)
- Line 5 calls the closure with argument **5**:
 - So body evaluated in environment with **x** $\mapsto$ **1**, **y** $\mapsto$ **5**

Lexical Scope via Closures

- We know the rule:
call evaluates function body in environment where function defined
 - And the model for implementing lexical scoping via closures
- So what's left? We already know everything!
 - See some (silly) examples to demonstrate lexical scope with higher-order functions
 - Discuss why **dynamic scope** is usually a Bad Idea™
 - See many powerful *idioms* with higher-order functions
 - Example: passing functions to iterators like `filter`

The Rule Stays the Same

- With higher-order functions, our semantics stays the same
 - Function body evaluated in environment where it was defined, extended w/ $\text{param} \mapsto \text{arg}$
- Nothing changes when we take / return functions
 - But “the environment” may not be the current toplevel environment
- Unintuitive at first, but makes first-class functions much more powerful!

Returning a Function

```
(* 1      *) let x = 1
(* 2      *) let f y =
(* 2 a *)    let x = y + 1 in
(* 2 b *)    fun q -> x + y + q
(* 3      *) let x = 3
(* 4      *) let g = f 4
(* 5      *) let y = 5
(* 6      *) let z = g 6
```

“Trust the rule”: Line 4 binds **g** to a closure:

- **Code**: take **q** and have body **x + y + q**
- **Environment**: **y** $\mapsto$ **4**, **x** $\mapsto$ **1**, ... (anything else in scope)
- So this closure will always add **9** to its argument
- So Line 6 binds **z** to **15**

Passing a Function

```
(* 1 *) let f g =  
(* 1 a *)   let x = 3 in  
(* 1 b *)   g 2  
(* 2 *) let x = 4  
(* 3 *) let h y = x + y  
(* 4 *) let z = f h
```

- “Trust the rule”: Line 3 binds **h** to a closure:
 - **Code**: take **y** and have body **x + y**
 - **Environment**: **x** $\mapsto$ **4**, **f** $\mapsto$ **<closure>**, ... (anything else in scope)
 - So this closure will always add **4** to its argument
- So Line 4 binds **z** to **6**
 - Note Line 1a can't affect anything! Can/should safely delete it.

Why Lexical Scope?

- **Lexical scope** : use environment where function was defined
- **Dynamic scope** : use environment where function is called
- In the early days of PL, folks pondered: *Which rule is better?*
 - Lexical scope is *almost always* the right default
- Let's consider 3 precise, technical reasons why
 - Not a matter of opinion!

Why Lexical Scope?

1. Functions meaning does not depend on variable names, only “shape”

```
let f y =  
  let x = y + 1 in  
  fun q -> x + y + q
```

```
let f g =  
  let x = 3 in  
  g 2
```

Example: Change **f** to use **w** instead of **x**

- **Lexical scope** : cannot change behavior of function
- **Dynamic scope** : depends on the environment of the caller

Example: Remove unused variables

- **Lexical scope** : variable unused, cannot change behavior
- **Dynamic scope** : some **g** may use **x** and depend on it being 3
 - WEIRD 😡

Why Lexical Scope?

2. Functions can be type checked and reasoned about where they are defined

```
let x = 1
let f y = (* f : int -> (int -> int) *)
    let x = y + 1 in
    fun q -> x + y + q
let x = "hi"
let g = f 4
let z = g 6
```

- Example: **dynamic scope** tries to add string and has unbound variable `y`

Why Lexical Scope?

3. Closures can easily store the data they need

```
let rec filter f =  
  fun xs ->  
    match xs with  
    | [] -> []  
    | x :: xs' ->  
      if f x then  
        x :: filter f xs'  
      else  
        filter f xs'  
  
let greater_than x =  
  fun y -> x < y
```

```
let is_positive =  
  greater_than 0  
  
let only_positives =  
  filter is_positive  
  
let all_greater (xs, n) =  
  filter (greater_than n) xs
```

Does Dynamic Scope Even Exist?

- Lexical scope is definitely the right default, seen in most languages
- Dynamic scope is occasionally convenient in some situations
 - Allows code to “just bind variables used in another function” to change behavior without passing parameters
 - Can be convenient, but also a nightmare 🤪
 - Some languages (including Racket!) have special “opt-in support”, but most do not
- If you squint, exception handling is similar to dynamic scope
 - **raise** **e** jumps to “most recent” (dynamically registered) handler
 - Does not need to be syntactically inside the handler!

When Things Evaluate

- We know:
 - Function body not evaluated until call
 - Function body evaluated every time function is called
 - In environment from when function was defined
 - Variable bindings evaluate right-hand-side once at binding time
 - NOT every time variable is used
- w/ closures, can avoid repeat computations that don't depend on args
 - Can make code run faster, but also interesting semantics

Recomputation

These both work and are equivalent

```
let all_shorter xs s =  
  filter (fun x -> String.length x < String.length s) xs  
  
let all_shorter' xs s =  
  let n = String.length s in  
  filter (fun x -> String.length x < n) xs
```

First version computes `String.length s` repeatedly (once per `x` in `xs`)

Second version computes `String.length s` once before filtering

- No new features! Just new use of closures :)

Fold

```
let rec fold_left f acc xs =  
  match xs with  
  | [] -> acc  
  | x :: xs' -> fold_left f (f acc x) xs'
```

- Another “hall-of-fame higher-order function” aka **fold** aka **reduce**
 - Accumulate answer by repeatedly applying **f** :
 - **fold_left f acc [v1; v2; v3]**
→ **f (f (f (f acc v1) v2) v3)**
 - **fold_left: ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a**
 - Contrast with **fold_right** (goes through list “the other way”)

Iterators

- **map**, **filter**, **fold** are similar to iteration patterns we see in imperative PLs
 - But not “built into” OCaml -- we can just define them ourselves!
 - “Just an idiom”! Power from combining elegant features.
- Separate “traversal” from “computation”
 - Allows reusing traversal code
 - Allows reusing computation code
 - Allows common way to talk about traversing data structures
 - Most collections provide **map**, **filter**, **fold** functions that “work the same way”

Group Work

```
type 'a tree =  
  | Leaf  
  | Node of 'a * 'a tree * 'a tree
```

Write a function `tree_map f tree` that with type
`('a -> 'b) -> 'a tree -> 'b tree` that applies `f` to every node in `tree`.

Example:

`map (fun x -> x + 1) (Node (3, Leaf, Node(1, Leaf, Node (2, Leaf, Leaf))))`
should return `Node (4, Leaf, Node (2, Leaf, Node (3, Leaf, Leaf)))`

```
(* ('a -> 'b) -> 'a list -> 'b list *)  
let rec map f xs =  
  match xs with  
  | [] -> []  
  | x :: xs' -> f x :: map f xs'
```